NIGHTCLUB CITY

Browser Game Reconstruction Blueprint

Reconstruction project brief

16 October 2026

Contents

1. Executive direction . . . 3
2. Evidence and confidence levels . . . 3
3. Target scope and exclusions . . . 5
4. Original game identity: visual and interaction targets . . . 5
5. Screen inventory . . . 6
6. Core gameplay loop . . . 6
7. Functional requirements . . . 7
8. Technical recommendation . . . 10
9. Asset reconstruction plan . . . 12
10. Detailed phase plan and acceptance gates . . . 13
11. Test plan . . . 14
12. Balancing and tuning file . . . 15
13. Deliverable definition . . . 15
14. Instructions for Claude Opus 5.5 . . . 15
15. Source list and research note . . . 16

Purpose: A build-ready specification for reconstructing the classic Nightclub City Facebook browser game from publicly available screenshots, walkthroughs, reviews, archived descriptions, and preservation research when original game files are unavailable.

Target: Original Facebook/browser Flash-era experience, reconstructed as a standalone modern browser game. Not Roblox. Not a generic nightclub tycoon.

Prepared: 16 October 2026
Companion file: `Claude_Opus_5_5_Master_Build_Prompt.md`

---

1. Executive direction

The goal is to make a playable reconstruction that captures the original game's visual identity, core interactions, management loop, UI, pacing, and social feeling as closely as the surviving evidence allows. The lack of original source code or assets means exact binary/asset-level restoration cannot be promised. The project must keep that distinction clear.

The browser game should feel like a preserved 2010-era social game: an isometric nightclub viewed from above at an angle, stylized cartoon patrons, neon dance-floor tiles, bars and seating, a visible DJ/player avatar, a compact dark-and-blue interface, money and club-rating counters, and a furniture/build mode that feels like the original rather than a modern generic dashboard.

The first deliverable is a **single-player playable vertical slice**. It must let a player enter a club, move around the scene, place and reposition a small catalog of furniture, watch guests arrive and behave, run a DJ set, serve drinks, earn tips, and save/reload the club. Only after this works reliably should the project add the larger progression, celebrities, parties, and social systems.

The reconstruction contract

1. Research before assumption: inspect source links and screenshots before deciding details.
2. Label evidence: use `CONFIRMED`, `INFERRED`, or `PROPOSED` in the project documentation and data comments.
3. Prefer fidelity over novelty: do not introduce unrelated tycoon mechanics, battle passes, pets, crafting, or Roblox-style systems.
4. When evidence is missing, use configurable values and document the uncertainty instead of presenting guesses as original facts.
5. Build with original replacement art unless the original assets are recovered and there is permission to use them. Do not use unlicensed commercial songs.
6. The first mode must work locally without requiring accounts, cloud services, payments, or live servers.
7. Do not claim the game is an exact original restoration; describe it as a reconstruction based on public evidence until original files and behavior can be verified.

---

2. Evidence and confidence levels

Use this legend in all design notes and implementation decisions:

- **CONFIRMED:** Supported by a historical walkthrough, review, screenshot, or preservation post.
- **INFERRED:** Strongly suggested by multiple images or descriptions, but a detail such as an exact formula, animation timing, or layout coordinate is unknown.
- **PROPOSED:** A practical implementation choice used to make the reconstruction playable. It must be configurable and should be revised if better evidence appears.
- **UNVERIFIED:** A claim or feature that has not yet been supported by a source.

Do not confuse `CONFIRMED feature exists` with `CONFIRMED exact implementation`. For example, popularity and luxury are visible in screenshots and guides, but the exact formula that calculates either score has not been recovered in this specification.

Historical sources to inspect first

1. **Gamezebo Nightclub City Walkthrough** — https://www.gamezebo.com/walkthroughs/nightclub-city-walkthrough/
   Documents the progress bar, popularity/luxury counters, DJ actions, worker list, drink-supply meter, money, click-to-inspect message box, menus, neighbors, avatar customization, furniture categories, DJ set durations, hiring bartenders and bouncers, and bouncer coverage.
2. **Gamezebo Nightclub City review page** — https://www.gamezebo.com/game/nightclub-city/
   Historical review/archive page. Use it to discover related coverage and terminology.
3. **GameYum Nightclub City review** — https://www.gameyum.com/facebook-games/70585-game-review-nightclub-city/
   Describes early gameplay, staff, celebrities, luxury, drink prices, decor, club expansion, and guest/customer loop.
4. **GameYum New Players Guide** — https://www.gameyum.com/facebook-games/73653-nightclub-city-new-players-guide/
   Additional early guide. Inspect it for more explicit rules and tutorial details.
5. **Elisa Knows: Nightclub City on Facebook** — https://www.elisaknows.com/2010/05/nightclub-city-on-facebook.html
   Contains a historical gameplay screenshot with the club, characters, HUD, cash, popularity, and luxury values.
6. **MobyGames: Nightclub City (2010)** — https://www.mobygames.com/game/74631/nightclub-city/
   Identifies the original browser version as an isometric management/business simulation and summarizes the venue-growth loop.
7. **Preservation thread seeking the original Facebook SWFs** — https://www.reddit.com/r/flash/comments/1wpf9ij/loc
   Posted 24 September 2026. The poster reports that a later official client and substantial source/data have been recovered and run offline, but the original Facebook Flash client is still missing. Known historic file names and hosts are listed in that post. Treat the report as a lead, not as proof that the files are publicly available.
8. **Historical screenshot of the avatar editor** — https://hotwebbackup.pixnet.net/blog/posts/7055411098
   Inspect screenshots for skin/hair options, character preview, category icons, save button, and UI structure.
9. **Historical furniture/build-mode screenshot** — https://adlieblogger.blogspot.com/2010/08/cheat-night-club-facebook.html
   Inspect screenshot for build mode, furniture categories, floor tiles, inventory, sell controls, cash and luxury indicators.
10. **Historical general review** — https://myfbapplication.blogspot.com/2010/07/nightclub-city.html
    Describes a fifteen-minute music mix and selectable broad genres, including electronic, hip-hop and pop. Cross-check against other guides.

Preservation search tasks

Before treating the original files as unobtainable, make one focused research pass through: - The current Reddit preservation thread above and any replies/linked threads. - Internet Archive / Wayback captures of `apps.facebook.com/nightclubcity/`, historic Nightclub City pages, and the historical game hosts named in the preservation thread. - Flashpoint Archive search and preservation forums. - MobyGames screenshots and contributor pages. - Old browser-game walkthroughs, blogs, YouTube videos, screenshots, and archived forum posts.

Known filenames noted by the preservationist include `NightclubMogulLoader_223.swf`, `_367.swf`, `_392.swf`, `_422.swf`, and `_455.swf`, with later paths resembling `/nlc/static/club/swf/v687/NightclubMogulLoader.swf`. Historical hosts listed include `dynamic00.nightclub-city.com`, `dynamic01.nightclub-city.com`, `content00.nightclub-city` `content01.nightclub-city.com`, and `titan.nightclub-city.com`. Do not assume those endpoints still work or probe systems aggressively. Search public archives and ask the preservationist for cooperation instead.

---

3. Target scope and exclusions

In scope

- Reconstruct the **Facebook/browser** version as the primary target, using the 2010–2012 browser UI and its visual language as the main reference set.
- Standalone, local-first browser experience that can run on a normal computer without Facebook integration.
- Isometric club scene, tile/furniture placement, avatars, guests, staff, music-set controls, drink inventory, tips/cash, popularity/luxury, club expansion, level progression, avatar dressing room, furniture shop, parties, celebrity booking/wall, and neighbor-club simulation.
- Desktop-first interaction resembling a mouse-based Flash game, with basic responsive behavior so the game is usable on smaller screens.
- Replace old social platform features with explicitly labeled local/demo alternatives until real online services exist.

Out of scope for the first playable build

- Roblox or any other platform-specific port.
- A generic 3D first-person nightclub.
- Real-world branded songs, artist recordings, original proprietary art, or original game binaries unless a lawful source and permission are established.
- Facebook login, friends API, purchases, ads, or real-money monetization.
- Complex multiplayer networking before the local single-player game is stable.
- Claiming that unverified mechanics or generated art are original assets.

Copyright and release boundary

This spec supports a technical reconstruction and preservation prototype. Public distribution of a game using Nightclub City branding, original art, source, music, or other protected materials may require authorization from the relevant rights holders. Until permissions are clear, keep the project clearly marked as an unofficial fan reconstruction, use original placeholder/replacement art, use royalty-free or commissioned music, and do not distribute recovered copyrighted binaries. Ask a qualified professional before a public commercial release.

---

4. Original game identity: visual and interaction targets

Visual identity

Historical screenshots suggest the following; confirm through source images before fixing exact dimensions:

- Isometric room viewed from above with visible floor tiles, walls, and an open front edge.
- Stylized, big-headed cartoon avatars with strong silhouettes, dark outlines/shadows, small limbs, expressive eyes, and varied hair, hats, clothes, and accessories.
- High-contrast nightclub palette: cyan/blue/purple illuminated floor tiles; dark charcoal/black walls and interface trim; colored neon accents; warm bar lighting; bright object highlights.
- Furniture drawn with pseudo-3D/isometric depth: bars, stools, sofas/booths, DJ equipment, speakers, plants, partitions, decor, and dance-floor tiles.
- A compact top HUD showing player progression, cash, popularity, and luxury; game actions and menus near the perimeter; a lower icon dock and contextual message/action area.
- Distinct Build/Decorate mode where the furniture catalogue occupies a lower panel and the club remains visible behind it.
- Avatar dressing-room overlay with central character preview, tabs/categories, palettes/options, randomize/dice action where confirmed by screenshots, and Save/Cancel-like action.

Rendering rule

Do not invent a fixed world tile size before measuring at least three clear reference screenshots. Implement grid geometry through configuration (for example, `tileWidth` and `tileHeight`) so it can be tuned later. Use a classic

2:1 isometric projection only as an initial hypothesis, then align tile edges, object footprints, and UI proportions against the reference images. Keep camera projection logic separate from world/game coordinates.

Interface rule

Avoid a modern dashboard look. Use compact, game-like panels, icon-driven actions, small labels, strong outlines, gradients/gloss only where supported by reference, and a blue/cyan lower toolbar. The original game should occupy most of the screen. Do not put every feature into a large sidebar.

---

5. Screen inventory

Create a screenshot/evidence sheet for every screen before declaring it complete.

Screen / state	Purpose	Build priority
Boot / loading	Load assets, show progress, handle missing data	P0
Main club view	Core play scene, guests, HUD, select object/guest	P0
Build/decorate	Shop catalogue, place/move/rotate/sell objects, save layout	P0
Context panel	Inspect guest, staff, avatar, furniture, bar or DJ	P0
DJ/music set	Select genre, start/refresh set, show time remaining, special actions	P0
Bar stock	Show remaining drinks and restock action	P0
Avatar dressing room	Appearance categories, preview, randomize where supported, save	P1
Worker management	Hire/assign bartender or bouncer, choose service area	P1
Level/progress view	Current level, total earnings/XP evidence, next unlocks	P1
Party planner	Party type, start time/status, visitor/guest event	P2
Celebrity booking / wall	Invite a celebrity; list past visits / regulars	P2
Neighbors / visit club	Browse demo clubs, visit in read-only or short-session mode	P2
Inventory / catalog pages	Browse locked/unlocked furniture, decorations, staff, equipment	P1
Settings / pause	Audio, simulation speed if development-only, reset save, credits	P1
Error / empty state	Graceful handling of invalid save or missing asset	P0

---

6. Core gameplay loop

Intended loop

1. Player opens their club and sees current cash, level/progress, popularity, luxury, DJ state and stock.
2. Guests enter because the club has a usable entrance, appropriate furniture/equipment, music and capacity.
3. Guests choose activities such as dancing, drinking, sitting, socializing, or leaving.
4. Positive experiences may lead to drink purchases and tips. Unhappy guests may complain or leave; troublemakers may trigger bouncer behavior.
5. Player uses DJ actions, restocks drinks and assigns workers to sustain service.
6. Player spends cash on furniture, tiles, bars, DJ equipment, upgrades and decor to increase luxury and capacity.
7. Better club quality and progression unlock more equipment and more valuable guests/activities.
8. Player saves and returns later to continue improving the club.

Evidence: Historical walkthroughs and reviews describe the club-decoration and management loop, DJ music actions, bar service, staff, customer happiness, tips, celebrities, and club expansion. Exact formulas and many timers remain unknown and must be treated as tuneable reconstruction parameters.

---

7. Functional requirements

7.1 Main club scene — P0

- Render a clear isometric room with tile grid, walls, entrance/front edge, object depth and shadows.
- Show the player/DJ avatar, placed club objects and animated NPC guests.
- Guests should not walk through walls or solid furniture. Use a lightweight tile navigation grid or equivalent path planner.
- Click an empty tile to inspect or enter placement mode; click an object/guest/staff member to open a contextual panel.
- Support zoom in/out and camera pan only if present in references or needed for large rooms; keep controls subtle.
- Add a small tutorial overlay for first-time users, with an option to dismiss it.
- Include a development-only debug toggle showing tile coordinates, object footprints, navigation cells, guest states and simulation events.

7.2 Build and furniture system — P0

- Open/close furniture catalogue without losing the visible club context.
- Group products into evidence-based categories such as floor tiles, DJ booths/equipment, bars, seating/booths, wall treatments and decoration. Do not force unsupported categories if reference images disagree.
- Display thumbnail, item name, price, required level, footprint, and any documented stat effect.
- Select item then place it on a valid tile; show translucent placement preview and clear valid/invalid state.
- Move existing items; rotate only if evidence suggests rotation existed. If rotation is uncertain, implement it behind a config flag and mark it proposed.
- Sell/remove an item after confirmation. Refund ratio must be configurable; do not assume a historical number.
- Persist object position, orientation, variant, room layer and purchase state.
- Enforce room bounds and collision/overlap rules using data-driven footprints.
- Furniture placements affect room score/capacity only through configurable object properties.
- Include undo/cancel for the active placement and avoid accidental purchases from a single miss-click.

7.3 Economy and progression — P0/P1

- Track cash as an integer or fixed-point numeric value; never allow negative money unless an explicit debt mechanic is evidenced.
- All price, tip, drink price, refill cost, unlock level and reward formulas live in config/data files, not hard-coded across components.
- Support income from drink sales and tips where the interaction is implemented.
- Purchases must be atomic: validate funds and unlock requirements, apply the purchase once, update inventory and persist; no double-charging on repeated clicks.
- Show affordability feedback and locked-item reason.

- Implement popularity and luxury as distinct values because both appear in original UI/references. Document their different effects. Avoid pretending their exact formulas are known.
- Level progression should be represented by a configurable XP/earnings curve. The original exact formula is not proven here. Store total earnings separately from current cash.
- Club expansion and item unlocks should have table-driven requirements. Avoid selecting arbitrary historical levels as facts until a source verifies them.
- Include a developer reset and a test fixture with known cash/level so QA can cover progression deterministically.

7.4 Guests and needs — P0

Implement guests as a finite-state machine. Initial states:

- `ENTERING`
- `CHOOSING_ACTIVITY`
- `WALKING_TO_DANCE_FLOOR`
- `DANCING`
- `THIRSTY`
- `QUEUING_FOR_BAR`
- `DRINKING`
- `SEEKING_SEAT`
- `SITTING_OR_SOCIALIZING`
- `HAPPY`
- `UNHAPPY`
- `TROUBLEMAKER` (probabilistic, disabled in early smoke test if it destabilizes flow)
- `LEAVING`

Guest properties should include stable ID, appearance seed, position, target tile, state, happiness (clamped 0-100), thirst, patience, spend tendency, guest class, visit start time, and last state transition time. Keep hidden stats configurable.

- Spawn rate should depend on a configurable guest-attraction score, available capacity and an upper performance cap.
- Guest selection should prefer valid nearby destinations and recover when a destination disappears.
- A guest should not get stuck forever. Add pathfinding timeout, alternate goal and safe despawn at entrance/exit.
- Guest happiness should be affected by configurable music/activity, drink availability, queue patience, comfort and service. These are proposed formulas until supported by evidence.
- Distinguish guest appearance from guest behavior data so adding art variants does not rewrite gameplay logic.
- Ensure the game remains readable with 30-50 active guests on a representative desktop; use a lower cap on weaker devices and expose it in config.

7.5 Bar, supplies and bartender — P0/P1

- Bars are placeable furniture with capacity, service speed, purchase requirements, and a drinks-per-stock parameter.
- Show a supply meter and numeric quantity in the UI.
- Historical walkthrough describes a refill action that adds 100 drinks at a time; use that as a confirmed baseline but make the amount data-driven.
- If stock is empty, guests should display a clear unhappy/leave/alternative behavior rather than buying nonexistent drinks.
- Bartender must be assigned to a bar or cluster of nearby bars; the historical guide notes that one bartender may not operate separated bars efficiently.
- Service actions should have a queue, serve duration, stock decrement, guest payment, bartender availability and UI animation.
- Use transactions so a single drink order cannot charge twice or subtract stock twice.

7.6 DJ system — P0

- The player's avatar acts as DJ in the base version.
- Implement genre selection. Historical sources mention broad styles including hip-hop, electronic and pop; verify exact UI/availability from screenshots and guides.
- A default DJ system is described as having a 15-minute set, with higher-level equipment potentially allowing longer sets. Keep duration in catalog/config rather than hard-coding.
- Display status, genre, time remaining and an obvious way to refresh/restart a completed set.
- Implement "Drop the Bass" as a one-click special action that triggers the behavior shown in the source guide (guests are driven toward the bar / thirst or purchasing behavior may rise); treat the precise effect and cooldown as tuneable.
- Implement "Dedicate Song"/microphone action by selecting an unhappy guest and boosting happiness, as described in the guide.
- Do not use original commercial recordings or copyrighted song snippets. Use silence, synthesized ambience or properly licensed placeholder loops during the reconstruction. Keep audio files pluggable.
- Timer should be based on timestamps, not frame count. On tab suspension, game should resume consistently rather than skipping or duplicating timers.

7.7 Staff and bouncer — P1

- Worker roster with role label and hire/assign/remove flow.
- Bartender: serves at a linked bar; speed may depend on furniture/staff parameters.
- Bouncer: assigned position or service area; detects a troublemaker within range and handles a configurable response. Manual "remove guest" action may be supported because historical guides describe it.
- Visual coverage/range indicator in staff placement mode.
- Initially use generated NPC staff, not a required real friend invite. Document this as a replacement for Facebook-friend-based hiring.

7.8 Avatar dressing room — P1

- Central preview of the player's cartoon avatar.
- Data-driven appearance options for body/gender presentation, skin tone, hair color, hairstyle, face/accessory details, tops, bottoms and shoes based on available references.
- Include randomize/dice option only if clearly supported by target screenshots.
- Save and cancel are explicit; preview edits must not silently persist.
- Parts need compatible anchors/layers so hair, face, clothing and accessories render in a stable order.
- If only replacement assets are available, use a consistent modular art style rather than mixing unrelated sprites.

7.9 Parties, celebrities and guest types — P2

- Party planner includes type, start/finish state, countdown, requirements and rewards. Historical descriptions list themes such as Disco Dance Party, Ladies Night and a charity/black-tie party; validate exact names before treating them as final.
- Celebrity booking should cost currency or another configurable resource and increase guest attraction/earnings as supported by evidence.
- Celebrity wall records who has visited and whether a guest is a repeat/regular. Exact rules must be verified; if unknown, use a documented proposed rule and allow tuning.
- To avoid unauthorized likenesses, use fictional placeholder VIPs unless licensed/verified original art and names are available. Public-facing release must not imply real artists endorse the project.
- Parties and celebrity booking come after core single-player simulation is stable.

7.10 Neighbors / social features — P2

- The historical game supported visiting neighbors/friends' clubs. Reconstruct this after the single-player club is robust.
- Phase 1 social substitute: a directory of locally seeded demo clubs that are immutable and read-only. A visit may simulate a short DJ session/reward, if supported by further historical evidence.
- Phase 2: optional hosted accounts and cloud save only after server trust and privacy design are complete.

- Never trust client-submitted cash, item prices, XP, or rewards in a future online build. Validate authoritative transactions on the server.
- Use fictional visitor names and synthetic clubs in the MVP; don't require Facebook or scrape personal information.

---

8. Technical recommendation

Recommended stack

- **TypeScript + Vite** for development and builds.
- **Phaser 3** for the 2D game scene, input, animation, cameras, asset loading and game loop.
- HTML/CSS overlay panels for menus, catalogues, labels, tooltips and accessible form controls where DOM is more convenient than canvas.
- **Vitest** for deterministic logic tests.
- ESLint/Prettier or equivalent basic lint/format scripts.
- **LocalStorage for a tiny early prototype; IndexedDB or versioned JSON export/import for more complete saves.** Keep persistence behind a `SaveRepository` interface so cloud storage can be added later.
- Do not add backend/auth/database frameworks until the offline prototype proves the gameplay loop. For future hosted multiplayer, evaluate a small API and a relational database after defining the server authority model.

Architecture boundaries

Separate the application into four layers:

1. **Rendering / scene:** projection, sprites, animations, tile highlighting, camera, effects.
2. **Simulation / domain:** guest state machines, pathfinding, DJ timers, bar queues, economy, score calculation, progression.
3. **Data / configuration:** furniture catalog, prices, footprints, visual metadata, unlocks, tuning values, appearance variants.
4. **UI / persistence:** modal panels, HUD updates, save/load, migration, reset/export.

Rendering must read from domain state; UI must send commands into domain services. Avoid having the canvas directly mutate cash or save files. Use a command/service boundary for purchases and placement.

Suggested repository structure

```
nightclub-city-reconstruction/
  README.md
  docs/
    RECONSTRUCTION_AUDIT.md
    EVIDENCE_REGISTER.md
    GAMEPLAY_SPEC.md
    DECISIONS.md
    ASSET_REGISTER.md
    TEST_PLAN.md
    BUILD_LOG.md
  public/
    assets/
      backgrounds/
      floors/
      furniture/
      avatars/
      guests/
      staff/
      ui/
      audio/
```

```
src/
  app/
  scenes/
    BootScene.ts
    PreloadScene.ts
    ClubScene.ts
  render/
    isoProjection.ts
    depthSort.ts
    placementPreview.ts
    animationController.ts
  domain/
    state/
    guests/
    pathfinding/
    economy/
    dj/
    bar/
    staff/
    progression/
    parties/
  data/
    furnitureCatalog.ts
    tuning.ts
    progression.ts
    appearanceCatalog.ts
  ui/
    hud/
    panels/
    modals/
  persistence/
    SaveRepository.ts
    localSaveRepository.ts
    migrations.ts
  debug/
tests/
  unit/
  integration/
scripts/
```

This is a proposed starting structure; Claude should adapt it to the existing project rather than blindly replacing a working codebase.

Key domain models (shape can evolve)

```ts
type Vec2 = { x: number; y: number };
type GridPoint = { col: number; row: number };

type FurnitureInstance = {
  id: string;
  catalogId: string;
  col: number;
  row: number;
  rotation: 0 | 90 | 180 | 270;
  variant?: string;
};
```

```ts
type ClubState = {
  schemaVersion: number;
  cash: number;
  totalEarned: number;
  level: number;
  popularity: number;
  luxury: number;
  roomWidth: number;
  roomHeight: number;
  furniture: FurnitureInstance[];
  inventory: Record<string, number>;
  staff: StaffState[];
  appearance: AvatarAppearance;
  music: DJState;
  stock: { drinks: number };
  savedAt: number;
};
```

Domain models should include stable IDs and `schemaVersion`; never save transient pathfinding nodes, render handles, or Phaser objects into player data.

---

9. Asset reconstruction plan

Asset workflow

1. Create a source/contact sheet from authentic public screenshots and videos. Record source URL, date accessed, screenshot crop, target screen, observed object, confidence and any licensing note.
2. Create a visual inventory: isometric floor tile, wall segment, bar, stool, couch/booth, DJ booth, speaker, light, plant/decor, entrance/door, NPC body/hair/outfit pieces, UI icon, meter, toolbar button, popup frame, currency/stat icon.
3. Prefer recovering genuine, lawfully usable original files through cooperation with preservationists. Do not assume public visibility means free reuse.
4. If original files cannot be used, create original replacement art that matches the broad visual language (shape, scale, palette, perspective and readability) without tracing/copying original sprites in a way that creates a derivative asset without permission.
5. Use a consistent sprite grid/anchor convention. Keep raw source files and exported game assets separated.
6. Maintain `ASSET_REGISTER.md` with `assetId`, role, file path, source or creator, license/permission, footprint, anchor, dimensions, animation frames, confidence and known limitations.
7. Build missing art iteratively. A gray-box prototype is acceptable in early phases if clearly identified as placeholder; it is not acceptable to call the visual reconstruction complete with colored rectangles.

Recommended asset-production order

- Phase A: tile, walls, entrance, placeholder floor, one bar, one DJ booth, one speaker, one stool/booth, one couch, player avatar and 6 guest variants.
- Phase B: a small but coherent furniture catalogue (12-20 objects) and 2-3 floor styles.
- Phase C: modular avatar wardrobe and guest variety.
- Phase D: improved lighting/effects, venue themes and additional furniture only after the placement pipeline is stable.

Sprite and layering conventions

- Define a logical ground anchor for every object.
- Sort draw order by projected ground position plus explicit layer overrides for walls/front-wall occlusion and ceiling/wall art.
- Shadows and floor light glows should not change hitboxes.

- Keep hitboxes and placement footprints as data independent from sprite pixel bounds.
- Use animation atlases or sprite sheets for idle/dance/walk cycles; avoid frame-by-frame large GIFs in the game loop.

---

10. Detailed phase plan and acceptance gates

Phase 0 — Evidence audit and target definition

Work: Read the listed sources; collect screenshots; record what is known, inferred and unknown; choose the browser build target; define screen reference sizes and a rough interaction map. Search the current preservation thread once and record whether files are available, inaccessible or not found.

Deliverables: `RECONSTRUCTION_AUDIT.md`, `EVIDENCE_REGISTER.md`, `GAMEPLAY_SPEC.md`, `DECISIONS.md`, initial screen/asset inventory.

Gate: Every major feature has at least one source or an explicit `PROPOSED` label. A target set of screenshots is selected. No implementation claim is called exact without evidence.

Phase 1 — Playable shell and isometric renderer

Work: Create Vite/TypeScript/Phaser app, boot/loading flow, club scene, isometric projection, tile highlighting, object rendering/depth sort, mouse selection, basic camera, HUD wireframe, responsive canvas sizing, debug overlay.

Gate tests: App starts with `npm install` + `npm run dev`; production build passes; a 10x10 or configurable room renders; click-to-grid conversion stays aligned at all corners; objects sort correctly; no console errors.

Phase 2 — Build mode and furniture catalogue

Work: Create data-driven furniture catalog and valid/invalid placement preview; purchase/placement; move; cancel; sell/remove; tile and footprint collision; catalogue category tabs; affordability and unlock UI.

Gate tests: Cannot place outside room or on occupied tile; no purchase when funds are insufficient; cancel does not charge; invalid moves restore original position; save/load preserves layout; currency transactions cannot double-apply.

Phase 3 — Guest simulation and economy

Work: Spawn guests, entrances/exits, pathfinding, guest needs/states, dance floor, bar queues, drink sale, tips, happiness, popularity/luxury effects via configurable functions; developer simulation controls.

Gate tests: At least 10 guests can path to valid targets; no guest is stuck indefinitely; bar stock decreases exactly once per sale; no negative cash from invalid transactions; stopping music or empty stock has understandable behavior; performance remains acceptable at configured guest cap.

Phase 4 — DJ, supplies, staff, main UI

Work: DJ genre and duration controls, set countdown, refresh/restart, Drop the Bass, Dedicate Song target action, drink refill, bartender assignment, bouncer area/target interaction, original-style HUD/context panel.

Gate tests: Timers remain correct when browser tab is hidden/resumed; special actions have feedback and cannot be spammed beyond configured cooldown; drink refill adds configured amount once; bartender only serves assigned bar; bouncer target selection works.

Phase 5 — Avatar, progression, saves and tutorial

Work: avatar editor, appearance catalogs, randomizer if evidence supports, player progression, furniture unlocks, room expansion, local save repository, schema migration, export/import/reset, first-run tutorial.

Gate tests: Appearance preview and cancel behave correctly; save/reload preserves all persistent data; old schema migration is tested; corrupt saves recover safely; reset requires confirmation; tutorial can be dismissed and does not return unexpectedly.

Phase 6 — Parties, celebrity wall, neighbors

Work: party planner, timed event lifecycle, placeholder VIP bookings, celebrity wall, demo neighbor directory and read-only visits. Keep all names/assets configurable so non-original placeholders can be swapped if licensed references become available.

Gate tests: Party start/end state is deterministic; rewards cannot be claimed twice; event resumes after reload without duplicate rewards; demo visits never mutate another club's state.

Phase 7 — Visual fidelity, accessibility, stability and release packaging

Work: Compare screenshots side-by-side, improve sprite scale and UI proportions, add subtle lighting and animation, optimize guest draw/pathfinding, keyboard/focus and readable contrast, error handling, README, save backup, browser support notes.

Gate tests: No critical console errors; build/lint/test pass; core flows work after refresh; no assets with unknown provenance are marked safe for public distribution; known inaccuracies are documented.

---

11. Test plan

Automated unit tests

- Isometric grid-to-screen and screen-to-grid transforms (round-trip within tolerance).
- Furniture footprints, collision and room bounds.
- Purchase affordability, buy/sell transaction invariants, zero/negative/decimal price handling.
- Guest state transition legality and pathfinding failure fallback.
- Bar stock decrement and refill amount.
- DJ set start, finish, refresh and hidden-tab timestamp behavior.
- XP/level-up boundary conditions with configurable curves.
- Save serialization/deserialization and schema migration.
- Party/celebrity reward idempotency.

Integration / manual tests

1. Start a new save and finish the tutorial.
2. Buy and place a floor tile, bar, seat and DJ booth.
3. Attempt invalid placement and verify no cash is lost.
4. Start a set and verify guests dance/act; use both DJ special actions.
5. Restock drinks, serve guests, see stock decrease and cash/tips rise.
6. Exhaust stock and verify guest behavior changes visibly.
7. Assign bartender and bouncer; verify they operate at the correct locations.
8. Save, refresh, and ensure club layout, appearance, money and progression persist.
9. Import an intentionally invalid save and recover gracefully.
10. Run 30-50 guests for several minutes with debug overlay and check for stuck NPCs or sustained performance problems.
11. Resize the browser and verify canvas and HUD remain aligned.
12. Compare selected screens against evidence screenshots and record deviations.

Quality bar

- No dead controls: every visible action works or is visibly disabled with a reason.
- No fake completion: placeholder art/features are labeled as placeholders.
- No hidden economy mutation from rendering code.
- No hard-coded balance formulas scattered across UI components.
- No data loss on normal refresh/reload.
- No broken layout at common desktop aspect ratios.
- All unresolved historical behavior remains documented and configurable.

---

12. Balancing and tuning file

Keep uncertain formulas inside a single, well-commented tuning configuration. Initial values below are **proposed starter values**, not a claim about original values:

```
export const tuning = {
  maxGuests: 40,
  guestSpawnIntervalMs: 3500,
  guestBaseHappiness: 65,
  guestBasePatienceMs: 45_000,
  barRefillAmount: 100,       // historical guide describes +100 per refill
  defaultDjSetMs: 15 * 60_000, // historical guide describes a 15-minute starter set
  djSpecialCooldownMs: 20_000,
  saveDebounceMs: 500,
  targetFps: 60,
};
```

The default DJ duration and drink refill amount have historical support in the listed walkthrough; the guest cap, spawn rate, happiness, patience, cooldown and save debounce are implementation suggestions. Do not use these as evidence of original tuning. Put tunable values in JSON/TS data and label each one.

---

13. Deliverable definition

At the end of each phase Claude should update `BUILD_LOG.md` with: - What was implemented. - Files changed. - Commands executed. - Test/build results. - Known limitations and evidence gaps. - Next exact recommended task.

The first public/demo milestone should include: - A runnable browser app with instructions. - Main club scene with coherent original-style art (not just UI wireframes). - Build mode and small furniture catalogue. - Guest movement/activity, bar sales, cash/tips, DJ set and special actions. - Local saving and reload. - Avatar editor foundation. - Automated tests for projection, placement, economy, guest transitions and save/load. - `README.md`, `EVIDENCE_REGISTER.md`, `ASSET_REGISTER.md`, `TEST_PLAN.md` and `BUILD_LOG.md`.

Before any wider feature work, all P0 acceptance gates must pass.

---

14. Instructions for Claude Opus 5.5

1. Read this entire blueprint and the companion master prompt before writing code.
2. Inspect the current working directory; preserve existing work if any. Do not assume the directory is empty.
3. Research the linked sources if browser/network access is available; if not, state exactly which sources you could not access. Do not invent source findings.
4. Create the evidence register and audit before finalizing gameplay formulas or art measurements.
5. Begin implementation immediately after the audit by building the P0 vertical slice. Do not stop after returning a plan.
6. Work in small, testable increments. After every phase, run the available tests and production build, fix regressions, and update `BUILD_LOG.md`.
7. Use placeholder data only where needed, clearly label it, and keep all uncertain values configurable.
8. Favor a faithful browser game and local-first operation over a backend-heavy architecture at the outset.
9. If a key detail cannot be verified, make a documented decision and continue rather than blocking the entire project with excessive questions.
10. Report honestly what runs and what does not. Never say the game is complete unless the acceptance tests demonstrate it.

---

15. Source list and research note

These are historical secondary sources and preservation leads, not authoritative access to the original code. Re-check URLs and page contents before relying on them. Do not copy long text from any source into the game. Record your own concise findings, link to the source, and use screenshots as visual evidence.

- Gamezebo walkthrough: https://www.gamezebo.com/walkthroughs/nightclub-city-walkthrough/
- Gamezebo archive/review: https://www.gamezebo.com/game/nightclub-city/
- GameYum review: https://www.gameyum.com/facebook-games/70585-game-review-nightclub-city/
- GameYum player guide: https://www.gameyum.com/facebook-games/73653-nightclub-city-new-players-guide/
- Elisa Knows screenshot post: https://www.elisaknows.com/2010/05/nightclub-city-on-facebook.html
- MobyGames title record: https://www.mobygames.com/game/74631/nightclub-city/
- Pixnet historical screenshot post: https://hotwebbackup.pixnet.net/blog/posts/7055411098
- Build-mode screenshot post: https://adlieblogger.blogspot.com/2010/08/cheat-night-club-facebook.html
- Older overview: https://myfbapplication.blogspot.com/2010/07/nightclub-city.html
- Preservation thread: https://www.reddit.com/r/flash/comments/1wpf9ij/looking_for_lost_nightclub_city_facebook_sw
- Internet Archive Flash collection: https://archive.org/download/swf-flash-games
- Flashpoint Archive search: https://flashpointarchive.org/search?advanced=true

Final guiding rule: build a faithful reconstruction of the *experience* from evidence, not an ungrounded generic replacement. Keep confidence levels visible, assets traceable, and each feature testable.